

CAMPUSEATS

Name	Id No.
Shivam Kumar	20251651083
Raghav Soni	20251651076
Aniket kumar	20251651019
Nikhil Singh Tomar	20251651065
Aman Tripathi	20251651018

Task 1

1. Place Order

- User selects items from a vendor's menu and submits an order request

2. Process Payment

- System handles financial transactions including card, UPI, and campus credits

3. Manage Inventory

- Vendor updates stock availability for menu items

4. Track Delivery

- User monitors real-time status of their order from preparation to delivery

5. Authenticate User

- System handles user registration, login, and session management

6. Rate & Review

- User provides feedback and ratings for vendors and delivery experience

7. Browse Menu

- User searches and views available items from campus vendors

8. Manage Vendor

- Administrator onboards, updates, or removes campus vendors

9. Send Notification

- System sends alerts about order status changes

10. Generate Reports

- Administrator views analytics and system performance metrics

Task 3: Service Contracts

1. Order Service Contracts

Operation	Input	Output	Errors
createOrder	userId, vendorId, items[{itemId, quantity}], deliveryAddress	orderId, status, estimatedDeliveryTime	INVALID_USER, INVALID_VENDOR, INVENTORY_UNAVAILABLE, PAYMENT_FAILED

getOrderStatus	orderId	status, timeline[{time, status}], currentLocation	ORDER_NOT_FOUND, UNAUTHORIZED_ACCESS
cancelOrder	orderId, reason	success, refundAmount	ORDER_NOT_FOUND, CANCELLATION_NOT_ALLOWED, ALREADY_DELIVERED
updateOrder	orderId, itemsUpdate, deliveryAddress	updatedOrder, newTotal	ORDER_NOT_FOUND, ITEMS_UNAVAILABLE, PAYMENT_GAP

2. Catalog Service Contracts

Operation	Input	Output	Errors
getMenuItems	vendorId, category, availabilityFilter	items[{id, name, price, available}]	VENDOR_NOT_FOUND
checkAvailability	itemId, quantity	available, estimatedWaitTime	ITEM_NOT_FOUND
updateInventory	itemId, quantityChange	newStockLevel	ITEM_NOT_FOUND, INVALID_QUANTITY
listVendors	location, cuisineType, ratingMin	vendors[{id, name, cuisine, rating, status}]	LOCATION_NOT_SERVICED

3. User Service Contracts

Operation	Input	Output	Errors
authenticateUser	email, password	userId, token, profile	INVALID_CREDENTIALS, ACCOUNT_LOCKED
registerUser	name, email, password, role	userId, verificationToken	EMAIL_EXISTS, INVALID_INPUT
getUserProfile	userId	profile {name, email, role, preferences}	USER_NOT_FOUND
updatePreferences	userId, preferences	updatedProfile	USER_NOT_FOUND

4. Payment Service Contracts

Operation	Input	Output	Errors
processPayment	orderId, amount, paymentMethod	transactionId, status, confirmationId	INSUFFICIENT_FUNDS, INVALID_CARD, GATEWAY_ERROR
refundPayment	transactionId, amount	refundId, status	TRANSACTION_NOT_FOUND, ALREADY_REFUNDED
validatePaymentMethod	paymentToken	isValid, cardType	INVALID_TOKEN, EXPIRED

5. Delivery Service Contracts

Operation	Input	Output	Errors
createDelivery	orderId, pickupLocation, dropoffLocation	deliveryId, estimatedTime, driverId	LOCATION_INVALID, NO_DRIVERS_AVAILABLE
trackDelivery	deliveryId	location, status, driverContact	DELIVERY_NOT_FOUND
updateDeliveryStatus	deliveryId, newStatus	updatedDelivery	DELIVERY_NOT_FOUND, INVALID_STATUS_TRANSITION

Task 4

Specify your central operation — placeOrder — in full: its inputs, outputs, error cases, and the internal details it must hide from callers.

1. Basic Information

Field	Value
Operation Name	placeOrder
Service	Order Service
Team Member	Shivam kumar

Visibility	Only logged-in users can call this
HTTP Method	POST
Endpoint	/api/orders

2. Purpose

The purpose of this operation is to complete a user's food order. This operation performs the following tasks:

- Validates if the user exists and is active
- Checks if the vendor is open and accepting orders
- Verifies if all ordered items are available in stock
- Processes the payment through the payment gateway
- Arranges delivery by assigning a delivery driver
- Confirms the order and saves it in the database
- Sends notifications to the user, vendor, and delivery driver

3. Inputs (Request Body)

Parameter	Type	Required
userId	string	Yes

vendorId	string	Yes
items	array	Yes
deliveryAddress	object	Yes
paymentMethod	object	Yes
scheduleFor	timestamp	No

Inside items –

Parameter	Type	Required
itemId	string	Yes
quantity	integer	Yes
specialInstructions	string	No

Inside **deliveryAddress** –

Parameter	Type	Required
building	string	Yes
room	string	No
landmark	string	No
instructions	string	No

Inside **paymentMethod** –

Parameter	Type	Required
type	string	Yes
token	string	Yes

4. Outputs

Success Response (201 Created):

Field	Type
orderId	string
status	string
estimatedDeliveryTime	timestamp
trackingId	string
receipt	object
vendorDetails	object
deliveryDetails	object

Inside **receipt object** –

Field	Type
subtotal	decimal
tax	decimal

deliveryFee	decimal
grandTotal	decimal

Failure Response (400/402/404 etc.):

Field	Type
errorCode	string
message	string
details	object

5. Error Cases

Error Code	HTTP Status	Message Example
USER_NOT_FOUND	404	User not found with ID: user_123
USER_INACTIVE	403	User account is inactive
VENDOR_NOT_FOUND	404	Vendor not found with ID: vendor_456
VENDOR_CLOSED	400	Vendor is currently closed
ITEM_NOT_FOUND	404	Item not found with ID: item_789
ITEM_UNAVAILABLE	400	Item is currently unavailable
INSUFFICIENT_STOCK	400	Only 3 items available. Requested: 5

PAYMENT_FAILED	402	Payment failed. Please try again
INSUFFICIENT_BALANCE	402	Insufficient campus credits
DELIVERY_UNAVAILABLE	503	No delivery drivers available
LOCATION_INVALID	400	Invalid delivery address
DUPLICATE_ORDER	409	Duplicate order detected
INVALID_INPUT	400	Invalid input: quantity must be greater than 0

6. Internal Details (Hidden from Callers)

The following details are internal implementation specifics that are hidden from API callers:

Step-by-Step Internal Process:

Step 1: Validate User

- Call User Service to verify if the user exists and is active
- If validation fails, return error: USER_NOT_FOUND

Step 2: Validate Vendor

- Call Catalog Service to verify if the vendor exists and is open
- If vendor is closed, return error: VENDOR_CLOSED

Step 3: Check and Reserve Items

- For each item in the order:
 - Call Catalog Service to check item availability

- If item is unavailable, return error: ITEM_UNAVAILABLE
- If item is available, reserve it for 15 minutes

Step 4: Calculate Total Amount

- Sum the price × quantity for all items
- Add tax (8%)
- Add delivery fee (₹20 flat)
- Calculate grand total

Step 5: Process Payment

- Call Payment Service to process the payment
- If payment fails, return error: PAYMENT_FAILED

Step 6: Create Delivery

- Call Delivery Service to arrange delivery
- Assign a nearby available driver
- Calculate estimated delivery time (ETA)

Step 7: Save Order

- Insert order record in the orders table
- Insert order items in the order_items table
- Insert initial status in the order_status_history table

Step 8: Send Notifications

- Send confirmation notification to the user (email/SMS)

- Send order notification to the vendor
- Send assignment notification to the driver

Step 9: Return Response

- Return orderId, receipt, and delivery details to the user

Task 6

TASK 1: CAPABILITY LIST

- 1. Place Order - User places food order from vendor**
- 2. Process Payment - System handles financial transactions**
- 3. Manage Inventory - Vendor updates stock availability**
- 4. Track Delivery - Monitor real-time order status**
- 5. Authenticate User - Login and registration functionality**

6. Rate & Review - User feedback on vendors and orders

TASK 2: SERVICE DESIGN

Services with Team Members:

Service	Team Member	Data Owned
-----	-----	-----
User Service	Shivam	Users, Profiles, Refresh Tokens
Order Service	Aman	Orders, Order Items, Status History
Catalog Service	Aniket	Vendors, Menus, Menu Items, Inventory
Payment Service	Raghav	Payments, Transactions
Delivery Service	Nikhil	Deliveries, Locations, Delivery Staff

Cross-Service Communications:

From Service	To Service	Operation Called
-----	-----	-----

Order Service (Aman)	User Service (Shivam)	validateUser
Order Service (Aman)	Catalog Service (Aniket)	checkAvailability
Order Service (Aman)	Payment Service (Raghav)	processPayment
Order Service (Aman)	Delivery Service (Nikhil)	createDelivery

TASK 3: SERVICE CONTRACTS

1. Order Service Contracts (Team: Aman)

Operation	Input	Output	Errors
createOrder	userId, vendorId, items[], deliveryAddress	orderId, status, estimatedDeliveryTime	
USER_NOT_FOUND, VENDOR_NOT_FOUND, ITEM_UNAVAILABLE			
getOrderStatus	orderId	status, timeline, location	ORDER_NOT_FOUND
cancelOrder	orderId, reason	success, refundAmount	ORDER_NOT_FOUND, ALREADY_DELIVERED

2. Catalog Service Contracts (Team: Aniket)

| Operation | Input | Output | Errors |

Operation	Input	Output	Errors
getMenuItems	vendorId, category	items[{id, name, price}]	VENDOR_NOT_FOUND
checkAvailability	itemId, quantity	available, waitTime	ITEM_NOT_FOUND
updateInventory	itemId, quantityChange	newStockLevel	ITEM_NOT_FOUND

3. User Service Contracts (Team: Shivam)

Operation	Input	Output	Errors
authenticateUser	email, password	userId, token, profile	INVALID_CREDENTIALS
validateUser	userId	valid, role, profile	USER_NOT_FOUND
registerUser	name, email, password, role	userId, verificationToken	EMAIL_EXISTS

4. Payment Service Contracts (Team: Raghav)

Operation	Input	Output	Errors
-----------	-------	--------	--------

| processPayment | orderId, amount, paymentMethod | transactionId, status | INSUFFICIENT_FUNDS,
GATEWAY_ERROR |

| refundPayment | transactionId, amount | refundId, status | TRANSACTION_NOT_FOUND |

5. Delivery Service Contracts (Team: Nikhil)

| Operation | Input | Output | Errors |

|-----|-----|-----|-----|

| createDelivery | orderId, pickupLocation, dropoffLocation | deliveryId, estimatedTime, driverId |
NO_DRIVERS_AVAILABLE |

| trackDelivery | deliveryId | location, status, driverContact | DELIVERY_NOT_FOUND |

TASK 4: FULL SPECIFICATION - placeOrder

Basic Information:

| Field | Value |

|-----|-----|

| Operation Name | placeOrder |

Service	Order Service
Team Member	Aman
Endpoint	POST /api/orders

Purpose:

Creates a complete food order by validating user, checking inventory,
processing payment, and creating delivery.

Inputs:

Parameter	Type	Required	Description
-----	-----	-----	-----
userId	string	Yes	Unique identifier of the user
vendorId	string	Yes	Unique identifier of the vendor
items	array	Yes	List of items being ordered
items[].itemId	string	Yes	Menu item ID
items[].quantity	integer	Yes	Quantity (must be > 0)
deliveryAddress	object	Yes	Delivery address details

| paymentMethod | object | Yes | Payment details |

Outputs (Success):

| Field | Description |

|-----|-----|

| orderId | Unique order identifier |

| status | "CONFIRMED" |

| estimatedDeliveryTime | Estimated delivery time |

| receipt.grandTotal | Total amount to be paid |

Error Cases:

| Error Code | When It Occurs |

|-----|-----|

| USER_NOT_FOUND | User ID does not exist |

| VENDOR_NOT_FOUND | Vendor ID does not exist |

ITEM_UNAVAILABLE	Item is out of stock
PAYMENT_FAILED	Payment processing failed
DELIVERY_UNAVAILABLE	No delivery drivers available

Internal Process (Hidden from Caller):

1. Validate user from User Service
2. Check inventory from Catalog Service
3. Reserve items for 15 minutes
4. Process payment from Payment Service
5. Create delivery from Delivery Service
6. Create order record in database
7. Send notifications to user and vendor

TASK 5: DATABASE SCHEMA

Tables by Service:

| Service | Team Member | Tables |

|-----|-----|-----|

| User Service | Shivam | users, user_profiles, refresh_tokens |

| Order Service | Aman | orders, order_items, order_status_history |

| Catalog Service | Aniket | vendors, categories, menus, menu_items, inventory |

| Payment Service | Raghav | payments, transactions |

| Delivery Service | Nikhil | deliveries, delivery_locations, delivery_staff |

TASK 6: SERVICE VALIDATION

Five Properties of a Service:

| Property | Meaning |

|-----|-----|

| Reachable | Can the service be accessed over network? |

| Self-Contained | Does the service own its own data? |

| Has Contract | Does the service have a defined API? |

| Independent | Can the service be deployed separately? |

| Loosely Coupled | Do changes affect other services? |

Validation Table:

| Service | Team Member | Reachable | Self-Contained | Has Contract | Independent | Loosely Coupled | Issues & Fixes

|

|-----|-----|-----|-----|-----|-----|-----|-----|

| User Service | Shivam | ☒ | ☒ | ☒ | ☒ | ☒ | No issues |

| Order Service | Aman | ☒ | ☒ | ☒ | ☒ | ☒ | Issue: Depends on other services. Fix: Add retry logic and circuit breakers |

| Catalog Service | Aniket | ☒ | ☒ | ☒ | ☒ | ☒ | No issues |

| Payment Service | Raghav | ☒ | ☒ | ☒ | ☒ | ☒ | Issue: Needs order details. Fix: Take all data as input |

| Delivery Service | Nikhil | ☒ | ☒ | ☒ | ☒ | ☒ | No issues |

Issues Summary:

1. Order Service (Aman) - Not Independent

Problem: Depends on other services for order completion

Fix: Add retry logic, circuit breakers, and timeouts

2. Payment Service (Raghav) - Partially Independent

Problem: Needs order details from other service

Fix: Take all required data as input parameters